

Concorrência em Java

Threads, sincronização e a API `java.util.concurrent` na prática

Apostila completa sobre programação concorrente em Java: da criação de threads aos executores, filas bloqueantes e streams paralelos, com exemplos aplicados a uma central de despacho de entregas.

CONTEÚDO DA APOSTILA

- | | | | |
|-----------|---|-----------|---|
| 01 | Threads, Processos e a Base da Concorrência | 14 | Prevenindo Deadlocks |
| 02 | A Classe Thread e a Prioridade das Threads | 15 | ExecutorService e Thread Pools |
| 03 | Criando Threads: extends Thread x implements Runnable | 16 | Callable, Future, submit e execute |
| 04 | run() x start(), Estados e Interrupção de uma Thread | 17 | Agendando Tarefas com ScheduledExecutorService |
| 05 | Memória Compartilhada, Time Slicing e o Java Memory Model | 18 | Coleções Sincronizadas e Concorrentes |
| 06 | Interleaving, Atomicidade e Thread-Safety | 19 | ArrayBlockingQueue e Filas Bloqueantes |
| 07 | A Palavra-chave volatile | 20 | CopyOnWriteArrayList e ConcurrentLinkedQueue |
| 08 | Sincronização: Métodos e Blocos synchronized | 21 | WorkStealingPool, ForkJoinPool e Parallel Streams |
| 09 | O Monitor Lock e a Sincronização Reentrante | 22 | Ordenação em Streams Paralelos |
| 10 | Produtor-Consumidor e o Problema do Deadlock | 23 | Reduce e Collect em Streams Paralelos |
| 11 | wait, notify e notifyAll | 24 | Classes Atômicas: Thread-Safety Sem Lock |
| 12 | java.util.concurrent.locks: Lock e ReentrantLock | 25 | WatchService: Monitorando o Sistema de Arquivos |
| 13 | Starvation, Livelock e Locks Justos | | |

Sumário

-
- 01 Threads, Processos e a Base da Concorrência**
O que é um processo, o que é uma thread e por que dividir o trabalho entre várias
-
- 02 A Classe Thread e a Prioridade das Threads**
A classe Thread por dentro, seus campos e a prioridade de agendamento
-
- 03 Criando Threads: extends Thread x implements Runnable**
As duas formas clássicas de criar uma thread e quando escolher cada uma
-
- 04 run() x start(), Estados e Interrupção de uma Thread**
Execução síncrona x assíncrona, o ciclo de vida e como interromper uma thread
-
- 05 Memória Compartilhada, Time Slicing e o Java Memory Model**
Heap x stack, fatiamento de tempo e as garantias do Java Memory Model
-
- 06 Interleaving, Atomicidade e Thread-Safety**
Por que operações simples não são atômicas e o que significa ser thread-safe
-
- 07 A Palavra-chave volatile**
Visibilidade de variáveis entre threads e os limites do modificador volatile
-
- 08 Sincronização: Métodos e Blocos synchronized**
O bloqueio de seções críticas com métodos e blocos synchronized
-
- 09 O Monitor Lock e a Sincronização Reentrante**
O monitor lock de cada objeto e por que a sincronização é reentrante
-
- 10 Produtor-Consumidor e o Problema do Deadlock**
Um cenário clássico de deadlock nascendo de uma flag compartilhada
-
- 11 wait, notify e notifyAll**
Como wait e notifyAll evitam espera ocupada e destravam o deadlock
-
- 12 java.util.concurrent.locks: Lock e ReentrantLock**
Locks explícitos, hold count e as vantagens sobre o monitor intrínseco
-
- 13 Starvation, Livelock e Locks Justos**
Threads famintas, locks que nunca se resolvem e a alternativa dos fair locks
-
- 14 Prevenindo Deadlocks**
Ordem global de locks e tryLock como estratégias de prevenção
-
- 15 ExecutorService e Thread Pools**
Delegando a criação e o ciclo de vida das threads a um pool gerenciado
-
- 16 Callable, Future, submit e execute**
Tarefas que devolvem valor, o contrato do Future e invokeAll x invokeAny
-
- 17 Agendando Tarefas com ScheduledExecutorService**
Executando tarefas atrasadas ou repetidas com o ScheduledExecutorService
-

-
- 18 Coleções Sincronizadas e Concorrentes**
Por que HashMap não é thread-safe e as alternativas sincronizadas e concorrentes
-
- 19 ArrayBlockingQueue e Filas Bloqueantes**
Um produtor e um consumidor trocando dados por uma fila de tamanho fixo
-
- 20 CopyOnWriteArrayList e ConcurrentLinkedQueue**
Listas e filas concorrentes sem bloqueio, para leituras ou inserções frequentes
-
- 21 WorkStealingPool, ForkJoinPool e Parallel Streams**
Divisão e conquista com o ForkJoinPool e o paralelismo dos streams
-
- 22 Ordenação em Streams Paralelos**
forEach x forEachOrdered e o custo de preservar a ordem em paralelo
-
- 23 Reduce e Collect em Streams Paralelos**
Redução imutável, redução mutável segura e agrupamento em paralelo
-
- 24 Classes Atômicas: Thread-Safety Sem Lock**
Atualizando contadores e flags sem lock com o pacote `java.util.concurrent.atomic`
-
- 25 WatchService: Monitorando o Sistema de Arquivos**
Reagindo a arquivos criados, modificados ou apagados em um diretório

01 Threads, Processos e a Base da Concorrência

O que diferencia um processo de uma thread, e por que isso importa para quem programa

Todo programa em execução é um **processo**: uma unidade isolada, com seu próprio espaço de memória, chamado de *heap*. Dois processos nunca enxergam o heap um do outro — é como se cada aplicação vivesse em sua própria casa, sem portas para a casa vizinha.

Dentro de um processo, porém, o trabalho pode ser dividido em **threads**: unidades de execução independentes que compartilham o mesmo heap. Toda aplicação Java nasce com pelo menos uma thread, a *main thread*, mas nada impede que o código crie e inicie outras threads para trabalhar em paralelo.

Criar uma thread é bem mais barato do que criar um processo inteiro — não é preciso duplicar memória, apenas reservar uma pequena pilha própria, chamada *thread stack*, usada para variáveis locais e chamadas de método. Só que essa economia tem um preço: como todas as threads de um processo compartilham o mesmo heap, elas podem interferir umas nas outras ao acessar os mesmos objetos.

```
// Central de despacho RotaExpress — sem threads extras,  
// cada pedido só pode ser processado depois do anterior terminar  
void despacharPedidosSequencial(List<Pedido> pedidos) {  
    for (Pedido pedido : pedidos) {  
        calcularRota(pedido); // tarefa demorada  
        notificarMotorista(pedido);  
    }  
}
```

Na RotaExpress, calcular a melhor rota para um pedido pode levar alguns segundos. Se isso acontecer sempre na thread principal, o sistema inteiro trava enquanto espera. É exatamente esse tipo de situação — tarefas demoradas, processamento de grandes volumes de dados, ou um servidor que precisa atender várias conexões ao mesmo tempo — que justifica usar múltiplas threads.

Esse comportamento de fazer mais de uma coisa ao mesmo tempo é o que chamamos de **concorrência**: partes diferentes do programa avançam de forma independente, sem que uma tarefa precise terminar por completo antes que a próxima comece.

REGRA — Processo x Thread

Um **processo** tem seu próprio heap; múltiplos **processos** nunca compartilham memória entre si. Já as **threads** de um mesmo processo compartilham o heap e têm apenas a stack como espaço exclusivo.

02 A Classe Thread e a Prioridade das Threads

Por dentro da classe `java.lang.Thread` e do valor de prioridade que ela expõe

Em Java, uma thread é representada pela classe **Thread**, que implementa a interface **Runnable**. Essa interface tem um único método abstrato, *run*, que carrega o código que a thread vai executar. Cada instância de Thread guarda, de forma encapsulada, um grupo de threads, um nome, uma prioridade, um status e um identificador único.

Uma Thread pode ser construída sem argumentos, ou recebendo uma instância de Runnable — nesse segundo caso, é o objeto Runnable quem fornece a implementação do método run que será executado.

Toda thread também carrega uma **prioridade**, um valor inteiro de 1 a 10 que sugere ao escalonador do sistema operacional o quanto essa thread deveria ser favorecida na hora de repartir tempo de CPU. A classe Thread expõe três constantes prontas para isso.

Constante	Valor	Significado
<code>Thread.MIN_PRIORITY</code>	1	Prioridade mais baixa
<code>Thread.NORM_PRIORITY</code>	5	Prioridade padrão de qualquer thread nova
<code>Thread.MAX_PRIORITY</code>	10	Prioridade mais alta

Threads com prioridade mais alta têm mais chance de ser escalonadas antes das de prioridade baixa — mas isso é só uma sugestão. O comportamento real varia conforme o sistema operacional e a implementação da JVM, então prioridade nunca deve ser usada como garantia de ordem de execução.

```
Thread motoristaPrioritario = new Thread(() -> calcularRota(pedidoUrgente));
motoristaPrioritario.setPriority(Thread.MAX_PRIORITY);
motoristaPrioritario.start();
```

ATENÇÃO — Prioridade não é garantia

Nunca escreva lógica de negócio que dependa da ordem de execução sugerida pela prioridade. Trate-a apenas como uma dica para o escalonador, não como uma regra que o sistema vai cumprir sempre.

03 Criando Threads: extends Thread x implements Runnable

Duas formas clássicas de descrever o trabalho de uma thread, com vantagens diferentes

Existem três caminhos para colocar código para rodar em uma thread separada: estender a classe Thread, implementar a interface Runnable, ou usar um *Executor* — este último é o assunto de um capítulo mais adiante. Por enquanto, vale entender as duas primeiras opções, que são a base de tudo.

Estendendo Thread, a nova subclasse sobrescreve o método run com a tarefa que a thread concorrente deve executar. Para usá-la, basta criar uma instância com o construtor sem argumentos e chamar o método start.

```
class MotoristaThread extends Thread {
    private final Pedido pedido;

    MotoristaThread(Pedido pedido) {
        this.pedido = pedido;
    }

    @Override
    public void run() {
        calcularRota(pedido);
        notificarMotorista(pedido);
    }
}

new MotoristaThread(pedido).start();
```

Essa abordagem dá acesso direto aos métodos e campos da própria Thread a partir da subclasse, e permite criar uma nova thread dedicada para cada tarefa. O problema é que, como Java só permite herança simples, a subclasse fica presa à Thread e não pode estender mais nenhuma outra classe — um acoplamento que costuma dificultar a manutenção.

Implementando Runnable resolve essa amarra: qualquer classe pode implementar a interface, mesmo já estendendo outra. A instância de Runnable é então passada para o construtor de Thread que a aceita, e o start continua sendo chamado normalmente para executar o código de forma assíncrona.

```
Runnable tarefaDespacho = () -> {
    calcularRota(pedido);
    notificarMotorista(pedido);
};
new Thread(tarefaDespacho).start();

// ou com uma referência de método
new Thread(RotaExpress::processarProximoPedido).start();
```

Como Runnable é uma interface funcional, qualquer expressão lambda ou referência de método serve como implementação — o que deixa o código bem mais enxuto. Em troca dessa flexibilidade, a classe que implementa Runnable perde o acesso direto aos métodos e campos da Thread a partir do próprio

método `run`.

Abordagem	Vantagem principal	Limitação principal
<code>extends Thread</code>	Acesso direto aos métodos da <code>Thread</code>	Não pode estender outra classe
<code>implements Runnable</code>	Classe livre para estender qualquer outra	Menos controle sobre a própria thread

DICA — Prefira `Runnable`

Na prática, implementar `Runnable` (ou usar diretamente uma lambda) costuma ser a escolha mais flexível e mais comum no dia a dia — especialmente porque, com `ExecutorService`, você nem precisa mais criar a `Thread` manualmente.

04 run() x start(), Estados e Interrupção de uma Thread

A diferença entre executar código na sua própria thread e disparar uma nova

Um dos erros mais comuns de quem está começando com threads é confundir **run()** com **start()**. Chamar **run** diretamente executa aquele código de forma síncrona, na thread atual — é como chamar qualquer outro método comum, sem nenhuma concorrência envolvida. Só a chamada a **start** é que efetivamente cria a nova thread e executa o código de forma assíncrona.

```
MotoristaThread t = new MotoristaThread(pedido);
t.run(); // roda no thread atual – nada de concorrência aqui
t.start(); // agora sim, roda em uma thread nova
```

Ao longo da vida, uma thread passa por estados bem definidos pelo enum `Thread.State`.

Estado	Significado
NEW	A thread foi criada, mas start ainda não foi chamado
RUNNABLE	A thread está executando (ou pronta para executar) na JVM
BLOCKED	A thread aguarda liberar um monitor lock
WAITING	A thread aguarda indefinidamente outra thread agir
TIMED_WAITING	A thread aguarda por um tempo limitado
TERMINATED	A thread encerrou sua execução

Quando uma thread precisa ser sinalizada para parar — por exemplo, um motorista que cancelou a corrida — o método correto é **interrupt()**. Ele não força a parada; apenas marca um sinalizador interno que o próprio código da thread deve verificar (ou que uma operação bloqueante, como **sleep**, transforma em uma `InterruptedException`).

```
Thread motorista = new Thread(() -> {
while (!Thread.currentThread().isInterrupted()) {
    buscarProximoPedido();
}
});
motorista.start();
// ... alguns segundos depois
motorista.interrupt();
```

ATENÇÃO — Interromper não é matar

interrupt() apenas sinaliza a intenção de parar. Se o código da thread nunca verificar `isInterrupted()` e não estiver bloqueado em um método sensível a interrupção, a thread simplesmente ignora o pedido e continua rodando.

05 Memória Compartilhada, Time Slicing e o Java Memory Model

Como threads enxergam a memória do processo, e por que isso cria riscos

Cada thread tem sua própria **stack**, usada para variáveis locais e chamadas de método — nenhuma outra thread tem acesso a ela. Mas toda thread também acessa o **heap** do processo, onde vivem os objetos e seus dados. É esse espaço compartilhado que permite a uma thread ler e alterar um objeto que outra thread também está usando, e é justamente aí que moram os problemas de concorrência.

O sistema operacional distribui o tempo de CPU entre as threads através de **time slicing**: cada thread recebe uma pequena fatia de tempo para avançar, e quando essa fatia acaba, ela cede o processador para outra, independentemente de ter terminado sua tarefa ou não. O problema é que, entre uma fatia e outra, o estado do heap compartilhado pode mudar — outra thread pode ter alterado exatamente o objeto que a primeira estava usando.

Para colocar ordem nesse cenário, existe o **Java Memory Model** (JMM): uma especificação que define regras sobre como e quando as alterações feitas por uma thread se tornam visíveis para as demais. Duas ideias centrais do JMM guiam boa parte do resto deste material:

Atomicidade de operações — poucas operações em Java realmente acontecem de forma indivisível, e é preciso saber identificar quais não são. **Sincronização** — o mecanismo que controla, de forma explícita, o acesso de múltiplas threads a um mesmo recurso compartilhado.

REGRA — Duas memórias, um heap

Toda thread Java tem sua própria stack, privada e isolada. Mas todas as threads de um mesmo processo compartilham um único heap — e é nesse espaço comum que a concorrência exige cuidado.

06 Interleaving, Atomicidade e Thread-Safety

Por que uma operação aparentemente simples pode ser interrompida no meio

Quando várias threads rodam ao mesmo tempo, suas instruções podem se sobrepor no tempo — esse fenômeno é chamado de **interleaving** (intercalação). A ordem real de execução entre threads distintas nunca é garantida: uma thread pode ser pausada no meio de uma operação para que outra avance, e retomar depois de onde parou.

Isso só é seguro quando a operação em si é **atômica** — ou seja, acontece por completo, de uma só vez, sem estado intermediário visível para outras threads. O problema é que operações que parecem uma única instrução no código podem, na verdade, se traduzir em vários passos separados na máquina virtual.

Operação	Exemplo	Por que não é atômica
Incremento	contador++	Ler, somar e gravar são três passos distintos
Decremento	--contador	O mesmo problema, na direção contrária
Atribuição de long/double	total = 100_000_000_000L	Pode ser gravada em duas operações de 32 bits em algumas JVMs

Se duas threads incrementam o mesmo contador ao mesmo tempo, cada uma pode ler o mesmo valor antigo antes que a outra grave o novo — e um dos incrementos acaba se perdendo. Esse tipo de interferência é justamente o que torna um trecho de código *não* thread-safe.

```
int entregasConcluidas = 0;

// se duas threads chamarem isto ao mesmo tempo, uma entrega pode
// não ser contabilizada — entregasConcluidas++ não é atômico
void registrarEntrega() {
    entregasConcluidas++;
}
```

Um objeto ou trecho de código é considerado **thread-safe** quando sua correção não é comprometida pela execução concorrente de várias threads — isto é, o resultado e o estado visível do programa continuam corretos independentemente de como as threads foram intercaladas. Operações realmente atômicas e objetos imutáveis são dois exemplos naturais de código thread-safe.

DICA — Desconfie do óbvio

Não assuma que uma linha de código é atômica só porque parece simples. Incrementos, decrementos e atribuições de long/double são os exemplos clássicos que enganam até programadores experientes.

07 A Palavra-chave volatile

Garantindo visibilidade — mas não atomicidade — de uma variável entre threads

Para ganhar desempenho, o sistema operacional pode manter, para cada thread, uma cópia em cache de valores lidos do heap. Isso significa que quando uma thread altera uma variável compartilhada, essa mudança pode ficar só no cache local da thread por um tempo, sem ser imediatamente refletida na memória principal — e sem que outras threads a enxerguem ainda. Esse é o chamado **erro de consistência de memória**.

O modificador **volatile**, aplicado a um campo de classe, resolve exatamente esse problema: ele obriga toda leitura e escrita daquela variável a acontecer diretamente na memória principal, nunca em um cache específico de thread. Isso garante que qualquer thread sempre enxergue o valor mais recente gravado por outra.

```
class ControleDespacho {  
    // sem volatile, outra thread poderia nunca ver esta mudança  
    private volatile boolean aceitandoNovosPedidos = true;  
  
    void encerrarTurno() {  
        aceitandoNovosPedidos = false;  
    }  
  
    void loopDeDespacho() {  
        while (aceitandoNovosPedidos) {  
            despacharProximoPedido();  
        }  
    }  
}
```

É importante entender os limites de volatile: ele resolve visibilidade, mas **não** torna uma operação atômica. Um incremento como contador++ continua não sendo seguro mesmo se contador for volatile, porque a operação ainda envolve ler, somar e gravar em passos separados.

Quando usar	Quando evitar
Uma flag de estado compartilhada, como um sinalizador de liga/desliga	Quando a variável é usada por uma única thread
Comunicação simples de um valor entre threads	Quando é preciso armazenar uma quantidade grande de dados

ATENÇÃO — volatile não substitui synchronized

volatile garante visibilidade de uma única variável, não atomicidade de uma sequência de operações. Para proteger uma seção crítica composta por mais de um passo, é preciso sincronização de verdade — assunto do próximo capítulo.

08 Sincronização: Métodos e Blocos `synchronized`

Bloqueando o acesso concorrente à seção crítica de um objeto

A **seção crítica** de um trecho de código é a parte que referencia um recurso compartilhado, como uma variável de instância. Quando todas as seções críticas de uma classe estão protegidas, dizemos que a classe é thread-safe. Em Java, essa proteção é implementada com a palavra-chave **synchronized**.

Um **método sincronizado** garante que diferentes invocações, sobre o mesmo objeto, nunca se intercalem. Enquanto uma thread executa um método sincronizado de um objeto, qualquer outra thread que tente invocar um método sincronizado daquele mesmo objeto é bloqueada e suspensa, até que a primeira termine. Ao sair do método, todas as alterações feitas ficam visíveis para as demais threads.

```
class ContadorEntregas {  
    private int total = 0;  
  
    public synchronized void registrarEntrega() {  
        total++; // agora protegido: só uma thread por vez executa isto  
    }  
  
    public synchronized int getTotal() {  
        return total;  
    }  
}
```

Já o **bloco `synchronized`** aplica a mesma trava a um trecho de código mais granular, em vez do método inteiro. Isso costuma ser a opção melhor na maioria dos casos, porque limita a sincronização exatamente à seção crítica, dando muito mais controle sobre o momento em que outras threads devem esperar.

```
void processarPedido(Pedido pedido) {  
    calcularRota(pedido); // não precisa de lock  
  
    synchronized (this) {  
        filaDeDespacho.add(pedido); // só esta parte é seção crítica  
    }  
  
    notificarMotoristaDisponivel();  
}
```

O bloco também pode travar em um objeto diferente do *this*, o que permite que threads acessando partes independentes de uma mesma instância não precisem bloquear umas às outras — por exemplo, sincronizar em cada fila de bairro separadamente, em vez de travar a central de despacho inteira.

REGRA — Seção crítica protegida = classe thread-safe

Uma classe só é considerada thread-safe quando **todas** as suas seções críticas estão sincronizadas de forma consistente — bastar proteger algumas delas ainda deixa brechas para interferência.

09 O Monitor Lock e a Sincronização Reentrante

O cadeado intrínseco de cada objeto Java e por que chamadas aninhadas não travam

Todo objeto em Java carrega, embutido, um **lock intrínseco** — também chamado de *monitor lock*. Uma thread adquire esse lock ao executar um método sincronizado da instância, ou ao usar a instância como alvo de um bloco `synchronized`. O lock só é liberado quando a thread sai do bloco ou método sincronizado — mesmo que isso aconteça por conta de uma exceção lançada.

Apenas **uma thread por vez** pode segurar o lock de um objeto. Enquanto isso, todas as outras threads que também queiram acessar o estado da instância por código sincronizado ficam bloqueadas, aguardando a vez de adquirir o lock.

Um detalhe importante: se o código já executado pela thread que detém o lock chamar outro método sincronizado da mesma instância — de forma aninhada — essa nova chamada **não bloqueia**. Como as duas chamadas partem da mesma thread, e essa thread já é dona do lock, a chamada aninhada simplesmente reaproveita o lock já adquirido. Esse comportamento é chamado de **sincronização reentrante**.

```
class CentralDeDespacho {  
    public synchronized void receberPedido(Pedido p) {  
        registrar(p);  
        atualizarEstoque(p); // chamada aninhada — não bloqueia  
    }  
  
    private synchronized void atualizarEstoque(Pedido p) {  
        // mesma thread, mesmo objeto: o lock já é dela  
    }  
}
```

Sem reentrância, uma thread poderia acabar bloqueada esperando um lock que ela mesma já possui — um impasse permanente. Esse mecanismo está embutido na própria linguagem e se baseia diretamente no monitor de cada objeto.

DICA — Reentrância evita autobloqueio

Graças à sincronização reentrante, um método sincronizado pode chamar outro método sincronizado do mesmo objeto sem medo de travar a própria thread que já detém o lock.

10 Produtor-Consumidor e o Problema do Deadlock

Como uma flag compartilhada, sem o cuidado certo, trava duas threads para sempre

Uma aplicação **produtor-consumidor** divide o trabalho em duas pontas: uma classe **produtora**, que gera dados, e uma classe **consumidora**, que lê e processa esses dados. Na RotaExpress, o produtor é o sistema que recebe novos pedidos, e o consumidor é a rotina que os despacha para os motoristas.

Imagine uma implementação ingênua, com uma flag booleana compartilhada indicando se há um pedido pronto para ser lido:

```
class RepositorioDePedidos {  
    private Pedido pedidoAtual;  
    private boolean temPedido = false;  
  
    void enviar(Pedido p) {  
        while (temPedido) { } // espera ocupada  
        pedidoAtual = p;  
        temPedido = true;  
    }  
  
    Pedido receber() {  
        while (!temPedido) { } // espera ocupada  
        temPedido = false;  
        return pedidoAtual;  
    }  
}
```

O método `receber` entra em um laço vazio enquanto a flag `temPedido` for falsa, esperando ela mudar de valor. Só que, para essa mudança acontecer, é o próprio método `enviar` que precisa rodar — e se os dois métodos estiverem sincronizados no mesmo objeto, a thread consumidora, presa nesse laço, nunca solta o lock para a thread produtora conseguir adquiri-lo e gravar o novo pedido.

O resultado é um **deadlock** clássico: uma thread fica girando indefinidamente em seu laço de espera, e a outra fica bloqueada tentando adquirir um lock que nunca vai ser liberado. Nenhuma das duas consegue avançar.

ATENÇÃO — Espera ocupada é uma armadilha

Um laço vazio esperando uma condição mudar (busy-waiting) não só desperdiça CPU — combinado com sincronização, é um dos jeitos mais comuns de criar um deadlock sem perceber. A solução está no próximo capítulo.

11 wait, notify e notifyAll

Colocando uma thread para dormir até que outra tenha novidades

Os métodos **wait**, **notify** e **notifyAll** pertencem à classe `Object` — ou seja, qualquer instância de qualquer classe pode chamá-los — e existem justamente para evitar o cenário de deadlock visto no capítulo anterior. Eles só podem ser chamados de dentro de um método ou bloco sincronizado na mesma instância.

Ao chamar **wait()**, a thread libera o lock que detém e entra em estado de espera, sem consumir CPU — bem diferente do laço vazio do capítulo anterior. Ela só volta a ser elegível para rodar quando alguma outra thread chamar **notify()** ou **notifyAll()** naquele mesmo objeto.

```
class CentralDeDespacho {
    private final List<Pedido> fila = new ArrayList<>();
    private static final int CAPACIDADE_MAXIMA = 20;

    public synchronized void receberPedido(Pedido p) throws InterruptedException {
        while (fila.size() >= CAPACIDADE_MAXIMA) {
            wait(); // libera o lock e aguarda espaço
        }
        fila.add(p);
        notifyAll(); // acorda quem está esperando um pedido
    }

    public synchronized Pedido despacharPedido() throws InterruptedException {
        while (fila.isEmpty()) {
            wait(); // libera o lock e aguarda um pedido
        }
        Pedido p = fila.remove(0);
        notifyAll(); // acorda quem está esperando espaço
        return p;
    }
}
```

Repare que a espera acontece dentro de um **while**, nunca de um simples **if**: quando a thread acorda, ela precisa reavaliar a condição, porque outra thread pode ter mudado o estado nesse meio-tempo, ou porque múltiplas threads podem acordar ao mesmo tempo com **notifyAll** e só uma delas deve seguir em frente.

Usar **notifyAll** em vez de **notify** é geralmente mais seguro: **notify** acorda apenas uma thread aleatória entre as que esperam, o que pode não ser a thread certa para a situação. **notifyAll** acorda todas, e cada uma reavalia sua própria condição no laço **while** antes de seguir.

REGRA — wait sempre dentro de um while

Nunca troque o **while** que envolve **wait()** por um **if**. Ao acordar, a thread deve sempre reconferir a condição — do contrário, ela pode agir sobre um estado que já mudou de novo.

12 java.util.concurrent.locks: Lock e ReentrantLock

Um controle mais explícito sobre a aquisição e liberação de um lock

A finalidade de um **lock** é controlar o acesso de múltiplas threads a um recurso compartilhado — exatamente o que o monitor lock intrínseco já faz. Mas o monitor lock tem limitações conhecidas: não há como testar se ele já está adquirido, não há como interromper uma thread bloqueada esperando por ele, é difícil de depurar, e ele é sempre exclusivo, sem opções de flexibilização.

Desde o JDK 5, o pacote **java.util.concurrent.locks** oferece a interface **Lock** — com implementações como **ReentrantLock** — dando bem mais controle sobre quando e como bloquear threads.

```
class CentralDeDespacho {
    private final Lock lock = new ReentrantLock();
    private final List<Pedido> fila = new ArrayList<>();

    void receberPedido(Pedido p) {
        lock.lock();
        try {
            fila.add(p);
        } finally {
            lock.unlock(); // sempre no finally!
        }
    }
}
```

Todo Lock mantém um **hold count**: a quantidade de vezes que a thread dona do lock o adquiriu. Na primeira aquisição, o hold count vai para um; se o lock for reentrante e a mesma thread adquiri-lo de novo, o contador sobe; cada chamada a unlock diminui o contador, e o lock só é realmente liberado quando ele chega a zero. É por isso que chamar unlock dentro de um bloco finally é indispensável, mesmo em código reentrante — esquecer isso mantém o lock preso para sempre.

Vantagem	O que ela permite
Controle explícito	Escolher exatamente quando adquirir e liberar, facilitando evitar deadlocks
Timeouts	Tentar adquirir um lock sem bloquear indefinidamente
Interrupção	Lidar com uma thread interrompida enquanto espera pelo lock
Depuração	Consultar quantas threads esperam, ou se uma thread já detém o lock

ATENÇÃO — unlock() sempre no finally

Diferente do bloco synchronized, que libera o lock automaticamente mesmo em caso de exceção, um Lock explícito só é liberado se você chamar unlock() — e isso precisa estar em um finally para acontecer em qualquer cenário.

13 Starvation, Livelock e Locks Justos

Quando uma thread nunca consegue a vez, ou quando duas threads dançam sem sair do lugar

Starvation (inanição) acontece quando uma thread nunca consegue obter o recurso de que precisa para avançar — geralmente porque outras threads, mais gulosas, ficam sempre à frente na fila. A aplicação como um todo continua rodando, e parte do trabalho é feito, mas uma thread específica fica perpetuamente sem vez.

Isso costuma aparecer quando várias threads disputam repetidamente o mesmo lock, em um laço contínuo. Como um `ReentrantLock` comum não é justo por padrão, nada impede que a mesma thread consiga readquirir o lock várias vezes seguidas, enquanto as demais ficam sempre de fora:

```
Lock lock = new ReentrantLock(); // injusto por padrão

void loopDeDespacho() {
    while (turnoAtivo) {
        lock.lock();
        try {
            processarProximoPedido(); // mantém o lock por um tempo
        } finally {
            lock.unlock();
        }
        // sem lock justo, o mesmo despachante pode vencer a
        // disputa pelo lock repetidamente, deixando os outros famintos
    }
}
```

Um **fair lock** (lock justo) resolve isso garantindo que todas as threads esperando por ele tenham a mesma chance de adquiri-lo — ao contrário do monitor lock padrão, que não dá nenhuma garantia de ordem. Um `ReentrantLock` pode ser criado como justo ou não.

```
// true = modo justo: threads são atendidas em ordem de chegada (FIFO)
Lock lockJusto = new ReentrantLock(true);
```

Quando uma thread pede acesso a um lock justo, ela entra em uma fila FIFO, e o lock é sempre concedido à thread na cabeça dessa fila — a que está esperando há mais tempo.

Benefícios de um fair lock

Evita starvation

Sistema mais previsível e fácil de depurar

Contrapartidas

Pode reduzir a performance em sistemas com muita disputa

Implementação mais custosa internamente

Já o **livelock** é um problema diferente: duas ou mais threads ficam reagindo continuamente umas às outras, cada uma respondendo à ação da anterior, sem que nenhuma delas jamais complete seu trabalho. É como dois motoristas em um corredor estreito que, educadamente, tentam ceder passagem um ao outro ao mesmo tempo — e os dois acabam se movendo para o mesmo lado repetidamente, sem nunca se cruzarem.

```
class CruzamentoEstreito {  
    private volatile String cedeAVez = null;  
  
    boolean tentarAtravessar(String motorista, String outroMotorista) {  
        if (outroMotorista.equals(cedeAVez)) {  
            cedeAVez = motorista; // educadamente cede a vez de novo...  
            return false; // ...e os dois nunca atravessam  
        }  
        cedeAVez = outroMotorista;  
        return true;  
    }  
}
```

Se os dois motoristas reagirem sempre no mesmo ritmo, cada um fica cedendo a vez ao outro indefinidamente — nenhuma thread está bloqueada, ambas continuam trabalhando, mas nenhuma progride de verdade.

Livelocks costumam ser difíceis de depurar, mas algumas práticas ajudam a evitá-los: não deixar threads checando continuamente o estado umas das outras, usar timeouts para não esperar indefinidamente, e introduzir alguma aleatoriedade para quebrar a simetria entre as threads envolvidas.

DICA — Aleatoriedade quebra o espelho

Quando duas threads reagem de forma perfeitamente simétrica uma à outra, um pequeno atraso aleatório antes de tentar de novo costuma ser suficiente para romper o ciclo de um livelock.

14 Prevenindo Deadlocks

Duas estratégias práticas para não deixar múltiplos locks travarem sua aplicação

Vale revisar os três problemas clássicos de uma aplicação multi-thread lado a lado, porque é fácil confundi-los:

Problema	Descrição
Deadlock	Duas ou mais threads ficam bloqueadas, cada uma esperando a outra liberar um recurso
Livelock	Duas ou mais threads ficam em loop contínuo, cada uma reagindo à ação da outra
Starvation	Uma thread nunca consegue obter os recursos de que precisa para avançar

Um segundo cenário clássico de deadlock — além da flag compartilhada já vista — acontece quando duas threads precisam adquirir **múltiplos recursos**, mas em ordens diferentes. Se a Thread A trava o Recurso 1 e tenta travar o Recurso 2, enquanto a Thread B trava o Recurso 2 e tenta travar o Recurso 1, as duas ficam presas esperando uma pela outra.

```
// Thread A: entrega e depois combustível – nesta ordem
synchronized (veiculo) {
synchronized (posto) { reabastecer(veiculo); }
}

// Thread B: combustível e depois entrega – ordem invertida!
synchronized (posto) {
synchronized (veiculo) { reabastecer(veiculo); }
}
```

Duas estratégias comuns ajudam a evitar esse tipo de deadlock. A primeira é estabelecer uma **ordem hierárquica** para os locks e garantir que todas as threads os adquiram sempre na mesma sequência — eliminando a espera circular. A segunda é trocar `synchronized` por **tryLock**, do Lock explícito: em vez de bloquear indefinidamente, a thread tenta adquirir o lock e, se não conseguir, pode desistir e tentar de novo mais tarde, sem travar.

```
if (lockVeiculo.tryLock()) {
try {
if (lockPosto.tryLock()) {
try {
reabastecer(veiculo);
} finally {
lockPosto.unlock();
}
}
} finally {
lockVeiculo.unlock();
}
}
```

REGRA — Ordem global de locks

A forma mais simples de evitar deadlocks entre múltiplos recursos é garantir que toda thread da aplicação sempre os adquira na mesma ordem combinada — nunca invertida.

15 ExecutorService e Thread Pools

Delegando a criação e o gerenciamento de threads a um serviço especializado

Gerenciar threads manualmente — nomear, priorizar, interromper, unir cada uma — é rudimentar e propenso a erro. À medida que uma aplicação cresce, esse controle manual vira uma fonte de disputa por recursos, overhead de criação de threads e problemas de escalabilidade. É para resolver isso que existe a interface **ExecutorService**, com várias implementações prontas no pacote `java.util.concurrent`.

Benefício	O que muda
Abstração de tarefas	Você pensa em tarefas a executar, não em threads a gerenciar
Thread pools	Reduz o custo de criar uma thread nova para cada tarefa
Escalonamento eficiente	Aproveita melhor os múltiplos núcleos do processador
Sincronização embutida	Menos erros de concorrência do que gerenciar threads à mão
Desligamento gracioso	Evita vazamento de recursos ao encerrar a aplicação

Criar, destruir e recriar threads o tempo todo tem um custo real. Um **thread pool** reduz esse custo mantendo um conjunto de threads já criadas, prontas para reaproveitar assim que terminam uma tarefa, em vez de descartá-las.

Um pool de threads tem três peças: as **worker threads**, pré-criadas e mantidas vivas durante a aplicação; a **fila de tarefas**, onde as submissões aguardam em ordem FIFO até uma thread ficar livre; e o **gerenciador do pool**, que distribui as tarefas entre as threads e garante a sincronização correta.

```
ExecutorService despachantes = Executors.newFixedThreadPool(4);

for (Pedido pedido : pedidosDoDia) {
    despachantes.execute(() -> calcularRota(pedido));
}

despachantes.shutdown(); // não aceita novas tarefas, espera as atuais
```

Classe	Comportamento	Método de fábrica
FixedThreadPool	Número fixo de threads	<code>newFixedThreadPool</code>
CachedThreadPool	Cria threads sob demanda; pool de tamanho variável	<code>newCachedThreadPool</code>
ScheduledThreadPool	Agenda tarefas para rodar uma vez ou em intervalos	<code>newScheduledThreadPool</code>
WorkStealingPool	Distribui tarefas por work-stealing entre as threads	<code>newWorkStealingPool</code>

Classe	Comportamento	Método de fábrica
ForkJoinPool	Especializado para tarefas do tipo ForkJoinTask	—

DICA — Use `ExecutorService` até para uma única thread

Mesmo quando só uma thread extra é necessária, um `SingleThreadExecutor` ainda vale mais a pena do que gerenciar uma `Thread` manualmente — o desligamento gracioso e a fila de tarefas embutida já valem o uso.

16 Callable, Future, submit e execute

Recebendo um valor de volta de uma tarefa executada em outra thread

A interface **Runnable** executa uma tarefa, mas não devolve nada. Já a interface **Callable** foi criada justamente para isso: seu método funcional devolve um valor, e ainda pode lançar uma exceção verificada.

Interface	Método funcional
Runnable	void run()
Callable	V call() throws Exception

Um `ExecutorService` aceita tarefas de duas formas: **execute**, que recebe um `Runnable` e não devolve nada; e **submit**, que aceita tanto `Runnable` quanto `Callable` e sempre devolve um **Future** — um placeholder para um resultado que ainda está sendo computado.

```
ExecutorService pool = Executors.newFixedThreadPool(2);

Future<Rota> futuro = pool.submit(() -> calcularMelhorRota(pedido));

// ... outro trabalho pode acontecer aqui enquanto a rota é calculada ...

Rota rota = futuro.get(); // bloqueia até o cálculo terminar
Rota comTimeout = futuro.get(3, TimeUnit.SECONDS); // ou espera até 3s
```

O método **get** só devolve o resultado quando o cálculo estiver completo — chamado antes disso, ele bloqueia a thread atual até a tarefa terminar. A versão sobrecarregada de `get` aceita um tempo limite, evitando ficar bloqueado para sempre caso algo dê errado.

Quando várias tarefas precisam ser submetidas de uma vez, o `ExecutorService` oferece dois métodos convenientes: **invokeAll** executa todas as tarefas e só retorna quando todas tiverem terminado, devolvendo uma lista de `Futures`; já **invokeAny** executa as tarefas, mas devolve assim que a primeira delas terminar, sem se importar se as demais falharam.

Característica	invokeAny	invokeAll
Tarefas executadas	Ao menos uma, a primeira a terminar	Todas as tarefas
Resultado	O valor da primeira a terminar (não é um <code>Future</code>)	Uma lista de <code>Futures</code> , uma vez que todas terminem
Quando usar	Quando basta uma resposta rápida entre várias opções	Quando todas as tarefas precisam terminar antes de seguir

17 Agendando Tarefas com ScheduledExecutorService

Executando uma tarefa depois de um atraso, ou repetidamente em intervalos

Quando uma tarefa precisa rodar depois de um certo atraso, ou repetidamente em intervalos regulares, o tipo certo é o **ScheduledExecutorService** — uma especialização do **ExecutorService** voltada exatamente para agendamento.

```
ScheduledExecutorService agendador = Executors.newScheduledThreadPool(1);

// roda uma única vez, depois de 10 segundos
agendador.schedule(() -> verificarPedidosAtrasados(), 10, TimeUnit.SECONDS);

// roda a cada 30 segundos, a partir de agora
agendador.scheduleAtFixedRate(
    () -> atualizarPosicaoDosMotoristas(), 0, 30, TimeUnit.SECONDS);

// espera 30s entre o FIM de uma execução e o INÍCIO da próxima
agendador.scheduleWithFixedDelay(
    () -> sincronizarComServidorCentral(), 0, 30, TimeUnit.SECONDS);
```

A diferença entre *scheduleAtFixedRate* e *scheduleWithFixedDelay* é sutil, mas importante: o primeiro tenta manter um ritmo fixo entre os **inícios** de cada execução, mesmo que uma delas demore mais que o esperado; o segundo garante um intervalo fixo entre o **fim** de uma execução e o início da próxima, o que evita execuções se acumulando quando uma tarefa atrasa.

Antes de existir o pacote `java.util.concurrent`, Java já tinha uma forma de agendar tarefas: a classe **Timer**, disponível desde o JDK 1.3. Ao ser criado, um **Timer** dispara uma única thread de segundo plano, responsável por executar instâncias de **TimerTask** nos horários agendados.

```
Timer timer = new Timer();

timer.scheduleAtFixedRate(new TimerTask() {
    @Override
    public void run() {
        verificarPedidosAtrasados();
    }
}, 0, 30_000); // início imediato, repete a cada 30 segundos (em milissegundos)

// timer.cancel(); // encerra a thread do timer quando não for mais necessário
```

O **ScheduledExecutorService** substitui o **Timer** com vantagens reais: suporta múltiplas threads em vez de apenas uma, se integra ao restante da API de concorrência, e é mais resiliente a falhas — se uma **TimerTask** lançar uma exceção não tratada, o **Timer** inteiro para de funcionar silenciosamente, enquanto um **ScheduledExecutorService** continua executando as demais tarefas.

DICA — Sempre desligue o agendador

Assim como qualquer `ExecutorService`, um `ScheduledExecutorService` precisa ser encerrado explicitamente com `shutdown()` quando a aplicação termina — caso contrário, suas threads continuam vivas e a JVM não encerra sozinha. O mesmo vale para um `Timer`, que só libera sua thread ao chamar `cancel()`.

18 Coleções Sincronizadas e Concorrentes

Por que *HashMap*, *ArrayList* e *HashSet* não são seguros em múltiplas threads

O *HashMap* — assim como *ArrayList*, *LinkedList*, *HashSet* e *TreeSet* — não é thread-safe. Ele não tem nenhuma sincronização interna, e não garante consistência de memória durante a iteração: se uma thread modificar o map enquanto outra o percorre, o comportamento é imprevisível, podendo lançar uma *ConcurrentModificationException* ou corromper a estrutura interna.

Classe	Ordenada?	Bloqueante?	Thread-safe?
HashMap	Não	Não	Não
TreeMap	Sim	Não	Não
ConcurrentHashMap	Não	Não	Sim
ConcurrentSkipListMap	Sim	Não	Sim
Collections.synchronizedMap	Depende do map original	Sim	Sim

Existem duas famílias de solução. Os **wrappers sincronizados**, obtidos pela classe utilitária *Collections*, protegem uma coleção comum atrás de um único lock que serializa todo o acesso — simples de aplicar em código já existente, mas pouco eficiente sob alta concorrência, já que só uma thread por vez pode acessar a coleção inteira.

```
Map<String, Motorista> motoristas =  
Collections.synchronizedMap(new HashMap<>());
```

Já as **coleções concorrentes**, como *ConcurrentHashMap*, usam técnicas mais sofisticadas — lock granular por partes da estrutura, ou algoritmos sem bloqueio — que permitem acesso concorrente seguro sem precisar de um único lock para tudo. Isso as torna, na prática, mais eficientes do que os wrappers sincronizados na maioria dos cenários.

```
Map<String, Motorista> motoristas = new ConcurrentHashMap<>();  
  
// quando a ordenação pelas chaves também importa:  
Map<String, Motorista> motoristasOrdenados = new ConcurrentSkipListMap<>();
```

Vale entender também o que acontece ao iterar sobre uma dessas coleções. Os iteradores de *HashMap*, *ArrayList* e companhia são **fail-fast**: se detectarem, durante a iteração, que outra thread modificou a estrutura, lançam imediatamente uma *ConcurrentModificationException* em vez de continuar com um estado inconsistente. Esse comportamento existe só para ajudar a detectar bugs cedo — nunca deve ser tratado como um mecanismo de controle de concorrência.

DICA — Prefira coleções concorrentes em código novo

Os wrappers sincronizados existem principalmente para adaptar código legado ao uso concorrente com o mínimo de mudança. Em código novo, as coleções do pacote `java.util.concurrent` quase sempre são a escolha melhor.

19 ArrayBlockingQueue e Filas Bloqueantes

Um produtor e um consumidor trocando dados por uma fila de tamanho fixo

Para um array de tamanho fixo, a opção mais usada é a **ArrayBlockingQueue**: uma fila FIFO que **bloqueia** em duas situações — quando uma thread tenta remover um elemento de uma fila vazia, e quando uma thread tenta inserir um elemento em uma fila já cheia. Ela é a base natural de um cenário produtor-consumidor: uma thread produtora insere itens em uma ponta, e uma ou mais threads consumidoras os retiram pela outra.

```
BlockingQueue<Pedido> filaDeDespacho = new ArrayBlockingQueue<>(5);

Runnable produtor = () -> {
    while (recebendoPedidos) {
        Pedido novo = aguardarProximoPedido();
        boolean aceito = filaDeDespacho.offer(novo, 5, TimeUnit.SECONDS);
        if (!aceito) {
            registrarPedidoAtrasado(novo); // fila cheia por 5s seguidos
        }
    }
};

Runnable consumidor = () -> {
    while (turnoAtivo) {
        Pedido proximo = filaDeDespacho.poll(5, TimeUnit.SECONDS);
        if (proximo != null) {
            despachar(proximo);
        }
    }
};
```

Existem várias formas de inserir um elemento, cada uma reagindo de um jeito diferente quando a fila está cheia:

Método	Bloqueia?	Retorna	Lança InterruptedException?
add(e)	Não	boolean	Não — lança IllegalStateException
offer(e)	Não	boolean	Não
offer(e, tempo, unidade)	Temporariamente	boolean	Sim
put(e)	Sim	void	Sim

E para remover um elemento, o comportamento varia da mesma forma quando a fila está vazia:

Método	Bloqueia?	Retorna	Remove da fila?
peek()	Não	item ou null	Não
poll()	Não	item ou null	Sim

Método	Bloqueia?	Retorna	Remove da fila?
poll(tempo, unidade)	Temporariamente	item ou null	Sim
remove()	Sim, se vazia	item (ou exceção)	Sim
take()	Sim, se vazia	item	Sim

Quando a fila enche e a estratégia não é simplesmente esperar ou descartar, o método **drainTo** permite esvaziá-la de uma vez em outra coleção — por exemplo, para registrar em log todos os pedidos pendentes antes de seguir em frente:

```
List<Pedido> pendentes = new ArrayList<>();
filaDeDespacho.drainTo(pendentes); // esvazia a fila inteira para a lista
```

REGRA — put/take bloqueiam, offer/poll não

Use put e take quando a thread pode esperar. Use offer e poll — com ou sem timeout — quando a thread precisa continuar seu trabalho mesmo que a operação não seja concluída de imediato.

20 CopyOnWriteArrayList e ConcurrentLinkedQueue

Listas e filas concorrentes sem bloqueio, para leituras ou inserções frequentes

Nem toda coleção concorrente precisa bloquear como a `ArrayBlockingQueue`. Para listas e filas, a escolha certa depende do padrão de leitura e escrita.

`CopyOnWriteArrayList` serve para cargas de trabalho com muita leitura e pouquíssima escrita — cadastros de configuração, ou uma lista de motoristas ativos que é lida o tempo todo, mas raramente muda. A cada modificação — `add`, `remove` ou `set` — uma **cópia nova** do array interno é criada, e a alteração é aplicada nessa cópia.

```
List<Motorista> motoristasAtivos = new CopyOnWriteArrayList<>();

motoristasAtivos.add(novoMotorista); // cria uma cópia nova do array interno

for (Motorista m : motoristasAtivos) { // leitura nunca bloqueia e nunca lança
    notificar(m); // ConcurrentModificationException
}
```

Isso garante que threads lendo a lista nunca sejam bloqueadas por uma escrita, e nunca vejam uma `ConcurrentModificationException` — elas sempre enxergam o array tal como estava no momento em que começaram a ler. Em compensação, cada escrita é relativamente cara, porque duplica o array inteiro.

Já **`ConcurrentLinkedQueue`** é indicada para o oposto: inserções e remoções frequentes, como em cenários de produtor-consumidor ou agendamento de tarefas em que não faz sentido bloquear a thread. É uma fila baseada em nós encadeados, sem capacidade máxima, que usa algoritmos sem lock para permanecer thread-safe.

```
Queue<Notificacao> filaDeNotificacoes = new ConcurrentLinkedQueue<>();

filaDeNotificacoes.offer(notificacao); // nunca bloqueia
Notificacao proxima = filaDeNotificacoes.poll(); // null se vazia, nunca bloqueia
```

Classe	Bloqueia?	Melhor cenário
<code>ArrayBlockingQueue</code>	Sim (capacidade fixa)	Produtor-consumidor com controle de vazão
<code>ConcurrentLinkedQueue</code>	Não	Inserção/remoção frequente, sem limite de tamanho
<code>CopyOnWriteArrayList</code>	Não	Muita leitura, pouquíssima escrita

DICA — Escolha pelo padrão de acesso, não pela familiaridade

A pergunta certa não é "qual estrutura eu já conheço", e sim "essa coleção vai ser mais lida ou mais escrita, e o consumidor pode esperar ou não" — a resposta aponta direto para a implementação certa.

21 WorkStealingPool, ForkJoinPool e Parallel Streams

Dividindo tarefas grandes em pedaços menores e balanceando a carga entre threads

O **work-stealing thread pool** existe para tarefas que podem ser divididas em pedaços independentes. Cada worker thread mantém sua própria fila de tarefas; quando uma thread termina o que tinha para fazer e sua fila esvazia, ela pode "roubar" tarefas do final da fila de outra thread ainda ocupada. Isso equilibra a carga de trabalho entre as threads e reduz o tempo ocioso.

O **ForkJoinPool** é a implementação de Java para esse modelo, baseada no algoritmo de *fork-join* — dividir para conquistar. A ideia é quebrar uma tarefa complexa em subtarefas menores, processá-las de forma independente e em paralelo, e depois combinar os resultados parciais na resposta final.

```
ExecutorService poolDeRoteirizacao = Executors.newWorkStealingPool();

List<Callable<Rota>> tarefas = pedidosDoDia.stream()
    .map(pedido -> (Callable<Rota>) () -> calcularMelhorRota(pedido))
    .toList();

List<Future<Rota>> rotas = poolDeRoteirizacao.invokeAll(tarefas);
```

Existem dois tipos principais de paralelismo. O **paralelismo de tarefas** divide um programa em tarefas menores que rodam concorrentemente, cada uma em sua própria thread ou núcleo de processador. Já o **paralelismo de dados** processa partes diferentes de um mesmo conjunto de dados ao mesmo tempo — é exatamente esse segundo modelo que os **parallel streams** exploram, usando por baixo dos panos o ForkJoinPool comum da aplicação.

```
double distanciaTotal = pedidosDoDia.parallelStream()
    .mapToDouble(RotaExpress::calcularDistanciaEmKm)
    .sum();
```

Os parallel streams oferecem melhor desempenho em máquinas com múltiplos núcleos, simplificam o código de processamento concorrente e distribuem o trabalho automaticamente entre as threads disponíveis. Mas paralelizar nem sempre compensa: criar e coordenar múltiplas threads tem um custo, que pode superar o ganho quando a coleção é pequena ou a operação já é muito rápida — os resultados parciais de cada thread ainda precisam ser sincronizados de volta em uma resposta única, e essa etapa também tem overhead.

ATENÇÃO — Nem sempre paralelo é mais rápido

Para coleções pequenas ou operações muito rápidas, o overhead de criar e sincronizar threads pode deixar um parallel stream mais lento do que a versão sequencial equivalente. Meça antes de assumir que paralelizar ajuda.

22 Ordenação em Streams Paralelos

O que sorted() garante, e por que forEach pode imprimir fora de ordem

A operação intermediária **sorted()** ordena os elementos do stream de acordo com o Comparator informado, e isso vale tanto para um stream sequencial quanto para um paralelo — o conjunto de elementos processado depois dela está, de fato, ordenado.

O detalhe está na hora de **consumir** esse stream ordenado. A operação terminal **forEach**, em um stream paralelo, não garante que os elementos sejam visitados na ordem de encontro do stream — cada worker thread reporta seu resultado assim que termina, então a ordem de impressão pode não bater com a ordem em que os elementos foram ordenados.

```
motoristas.parallelStream()  
  .sorted(Comparator.comparing(Motorista::getNome))  
  .forEach(System.out::println); // ordem de impressão não é garantida
```

Para forçar a operação terminal a respeitar a ordem de encontro do stream, mesmo em paralelo, existe **forEachOrdered**. Ele garante a ordem correta, mas ao custo de sincronização extra, o que reduz parte do ganho de desempenho do processamento paralelo.

```
motoristas.parallelStream()  
  .sorted(Comparator.comparing(Motorista::getNome))  
  .forEachOrdered(System.out::println); // respeita a ordem do stream ordenado
```

REGRA — **forEach** para performance, **forEachOrdered** para ordem

Use **forEach** comum quando a ordem de processamento não importa — como em agregações e somas. Use **forEachOrdered** apenas quando o resultado precisa sair na ordem esperada, ciente de que isso reduz parte do ganho do paralelismo.

23 Reduce e Collect em Streams Paralelos

Redução imutável, redução mutável segura e agrupamento em paralelo

A operação **reduce** combina os elementos de um stream em um único resultado. Na forma de dois argumentos — uma identidade e um acumulador — ela funciona bem em paralelo quando o resultado é um valor **imutável**, como somas e concatenações simples de String.

```
int totalDeItens = pedidosDoDia.parallelStream()
    .mapToInt(Pedido::getQuantidadeItens)
    .reduce(0, Integer::sum);
```

O problema aparece quando se tenta usar **reduce** para uma **redução mutável** — acumulando resultados dentro de um único objeto mutável compartilhado, como um `StringJoiner`. Como o stream pode processar pedaços em threads diferentes, mais de uma thread pode acabar mutando o mesmo objeto ao mesmo tempo, se ele não for thread-safe, causando exceções ou resultados corrompidos.

Para esse caso, o Java oferece **collect**, feito especificamente para redução mutável. Em vez de uma identidade fixa, **collect** recebe um *fornecedor* que cria um **novο** recipiente para cada trecho do processamento paralelo — nenhuma thread nunca divide um recipiente mutável com outra; a combinação só acontece no final, entre os recipientes já prontos.

```
// Collectors.joining já usa collect por baixo, de forma segura
String placasDoTurno = motoristas.parallelStream()
    .map(Motorista::getPlaca)
    .collect(Collectors.joining(", "));
```

ATENÇÃO — Nunca compartilhe um objeto mutável em um reduce paralelo

Se a redução precisa acumular em um objeto mutável (`StringBuilder`, `StringJoiner`, uma lista), use **collect** ou um **Collector** pronto como `Collectors.joining` — nunca um **reduce** manual reaproveitando o mesmo objeto como identidade entre threads.

Outro ponto de atenção é o agrupamento. Por padrão, **Collectors.groupingBy** devolve um `HashMap` comum — mesclar vários `HashMap`s parciais, um de cada thread, é uma operação cara. Para um stream paralelo, **Collectors.groupingByConcurrent** é bem mais eficiente: ele devolve um `ConcurrentHashMap`, que todas as threads podem atualizar diretamente, sem que seja preciso mesclar mapas separados no final.

```
Map<String, Long> pedidosPorBairro = pedidosDoDia.parallelStream()
    .collect(Collectors.groupingByConcurrent(
        Pedido::getBairro, Collectors.counting()));
```

As operações terminais **toArray** e **toList** seguem essa mesma lógica de coleta segura: cada segmento paralelo monta sua própria parte, e o framework de streams combina tudo em uma única coleção final, sem que o código-cliente precise se preocupar com sincronização.

DICA — `groupingByConcurrent` em streams paralelos

Sempre que agrupar dados a partir de um `parallelStream`, prefira `groupingByConcurrent` a `groupingBy` — evita o custo de mesclar múltiplos `HashMap`s parciais no fim do processamento.

24 Classes Atômicas: Thread-Safety Sem Lock

Atualizando contadores e flags sem lock com o pacote `java.util.concurrent.atomic`

Um gerador de identificadores é um exemplo clássico de seção crítica pequena, mas fácil de esquecer de proteger:

```
class GeradorDeId {  
    private int proximoId = 0;  
  
    int getNextId() {  
        return ++proximoId; // não é atômico!  
    }  
}
```

Se várias threads chamarem `getNextId` ao mesmo tempo, mais de uma pode ler o mesmo valor antes de qualquer uma gravar o incremento — gerando IDs duplicados e, na prática, menos identificadores únicos do que o esperado.

A correção mais direta é sincronizar o método, como visto nos capítulos anteriores. Mas para esse tipo de contador simples, o pacote **`java.util.concurrent.atomic`** resolve o mesmo problema de um jeito mais leve: um pequeno conjunto de classes que atualiza uma única variável de forma atômica, **sem precisar de lock algum**.

```
class GeradorDeId {  
    private final AtomicInteger proximoId = new AtomicInteger(0);  
  
    int getNextId() {  
        return proximoId.incrementAndGet(); // atômico, sem lock  
    }  
}
```

Por não usar lock, uma classe atômica costuma ter desempenho melhor do que sincronizar um método inteiro, especialmente em aplicações de alto throughput com muitas threads disputando o mesmo contador.

Elemento único	Versão em array
AtomicBoolean	—
AtomicInteger	AtomicIntegerArray
AtomicLong	AtomicLongArray
AtomicReference	AtomicReferenceArray

```
AtomicInteger entregasConcluidas = new AtomicInteger(0);

void registrarEntrega() {
    entregasConcluidas.incrementAndGet(); // atômico, sem lock
}
```

DICA — Lock-free para contadores simples

Sempre que a necessidade se resumir a atualizar uma única variável — um contador, uma flag, uma referência — vale considerar uma classe atômica antes de recorrer a `synchronized` ou a um `Lock` explícito.

25 WatchService: Monitorando o Sistema de Arquivos

Reagindo a arquivos criados, modificados ou apagados em um diretório

O **WatchService** é um serviço especial que observa objetos registrados em busca de eventos e mudanças. Um caso de uso típico: a RotaExpress recebe pedidos em lote através de arquivos depositados em uma pasta, e o WatchService pode monitorar essa pasta para processar cada novo arquivo assim que ele chega, sem precisar ficar checando o diretório em um laço manual.

```
WatchService watcher = FileSystems.getDefault().newWatchService();
Path pastaDePedidos = Paths.get("/pedidos/entrada");
pastaDePedidos.register(watcher,
    StandardWatchEventKinds.ENTRY_CREATE,
    StandardWatchEventKinds.ENTRY_MODIFY,
    StandardWatchEventKinds.ENTRY_DELETE);

while (true) {
    WatchKey chave = watcher.take(); // bloqueia até haver um evento
    for (WatchEvent<?> evento : chave.pollEvents()) {
        processarNovoArquivo((Path) evento.context());
    }
    chave.reset();
}
```

Um objeto *Watchable*, como um diretório, é registrado junto ao WatchService, informando quais tipos de evento interessam. Quando eventos acontecem nesse objeto, eles são enfileirados no serviço, e uma thread consumidora pode retirá-los da fila com **poll** ou **take**.

A **WatchKey** retornada pelo registro passa por estados bem definidos: começa *ready* (pronta); ao detectar um evento, torna-se *signalled* (sinalizada) e é enfileirada no serviço para ser recuperada; permanece sinalizada — acumulando novos eventos detectados nesse meio-tempo, sem entrar na fila de novo — até que seu método **reset** seja chamado, o que a devolve ao estado *ready*, pronta para ser sinalizada outra vez.

O WatchService não se limita a arquivos e diretórios: a mesma ideia se aplica a outros tipos de recurso observável, como bancos de dados, filas de mensagens ou regiões de memória compartilhada, sempre que uma implementação de Watchable estiver disponível para eles.

REGRA — Sempre chame reset()

Esquecer de chamar `reset()` em uma `WatchKey` a mantém presa no estado sinalizado — ela nunca mais será reenfileirada, e o código para de receber novos eventos daquele recurso, mesmo que eles continuem acontecendo.

Fim de apostila

Este material percorreu a concorrência em Java de ponta a ponta: threads e seu ciclo de vida, sincronização com `synchronized` e `Lock`, os problemas clássicos de deadlock, livelock e starvation, a API `java.util.concurrent` — de `ExecutorService` a filas e listas concorrentes — e o paralelismo de dados com `ForkJoinPool`, `parallel streams` e classes atômicas.

Esses conceitos sustentam qualquer aplicação Java que precise lidar com múltiplas tarefas ao mesmo tempo — de servidores web a pipelines de processamento de dados.